Sistemas Operativos II

Práctica 2 — Informe 2

Sincronización de procesos con semáforos

Asignatura:	Sistemas Operativos II
Titulación:	Grado en Ingeniería Informática
Curso:	2024/25
Fecha:	10 de marzo de 2026

Universidad de Santiago de Compostela

Índice

2. Informe 2 — Solución con semáforos POSIX **2**
2.1. Descripción de la solución 2
2.2. Cómo ejecutar los programas 2
2.3. Protocolo de sincronización 2
2.4. Fases de velocidad 3
2.5. Resultados observados 4
2.6. Gestión de semáforos en el kernel 4
2.7. Conclusiones 5

2. Informe 2 — Solución con semáforos POSIX

2.1. Descripción de la solución

La solución clásica al problema del productor-consumidor utiliza tres semáforos POSIX que abordan las tres causas de error del apartado anterior:

Semáforo	Val. inicial	Propósito
`VACIAS`	`N = 10`	Número de huecos libres. El productor hace `sem_wait` antes de insertar; se bloquea si el buffer está lleno.
`LLENAS`	`0`	Número de elementos disponibles. El consumidor hace `sem_wait` antes de retirar; se bloquea si el buffer está vacío.
`MUTEX`	`1`	Exclusión mutua en la región crítica (acceso a `buf` y `top`).

2.2. Cómo ejecutar los programas

```
gcc -o prod_sem prod_sem.c -pthread
gcc -o cons_sem cons_sem.c -pthread

# Terminal 1:
./prod_sem
# Terminal 2:
./cons_sem
```

El productor crea e inicializa los semáforos con `O_CREAT`. El consumidor los abre con `sem_open("NOMBRE", 0)`, reintentando hasta que el productor los haya creado.

2.3. Protocolo de sincronización

Productor

```
for (int iter = 0; iter < MAX_ITER; iter++) {
    char item = produce_item();   // FUERA de la region critica

    sem_wait(vacias);   // bloquear si buffer lleno
    sem_wait(mutex);    // entrar en region critica
    /* ====== REGION CRITICA ====== */
    insert_item(item);
    int val_top = shm->top; // capturar top dentro de la RC
```

```
    /* ============================ */
    sem_post(mutex);     // salir de region critica
    sem_post(llenas);    // avisar al consumidor

    sleep(fase_actual); // FUERA de la region critica
}
```

Listing 1: Esquema del bucle del productor (`prod_sem.c`)

Consumidor

```
for (int iter = 0; iter < MAX_ITER; iter++) {
    sem_wait(llenas);    // bloquear si buffer vacio
    sem_wait(mutex);     // entrar en region critica
    /* ====== REGION CRITICA ====== */
    char item = remove_item();
    int val_top = shm->top; // capturar top dentro de la RC
    /* ============================ */
    sem_post(mutex);     // salir de region critica
    sem_post(vacias);    // avisar al productor

    consume_item(item); // FUERA de la region critica
    sleep(fase_actual); // FUERA de la region critica
}
```

Listing 2: Esquema del bucle del consumidor (`cons_sem.c`)

Orden crítico de las llamadas

`sem_wait(condición)` debe preceder siempre a `sem_wait(mutex)`. Si se invirtiese el orden, un proceso podría retener el `MUTEX` mientras espera en el semáforo de condición, bloqueando al otro proceso indefinidamente (*interbloqueo*).

2.4. Fases de velocidad

El bucle de 80 iteraciones tiene tres fases con `sleep()` situados **fuera** de la región crítica:

Iteraciones	Prod.	Cons.	Efecto esperado
0 – 29	0 s	2 s	Buffer lleno. Productor bloqueado en `sem_wait(vacias)` cuando `top = N`.
30 – 59	2 s	0 s	Buffer vacío. Consumidor bloqueado en `sem_wait(llenas)` cuando `top = 0`.
60 – 79	rand 0–3 s	rand 0–3 s	Comportamiento mixto; `top` oscila entre 0 y 4.

2.5. Resultados observados

La salida de `prod_sem` muestra `top_tras_insert = 10` de forma sostenida durante las iteraciones 10–39, lo que confirma que el productor avanza sin esperar al consumidor (fase 1) y el buffer se mantiene lleno. En la fase 2 (iter 30–59) `top_tras_insert` cae a 1, evidenciando que el consumidor vacía el buffer casi en tiempo real. En la fase aleatoria los valores de `top` oscilan entre 1 y 4, reflejando el comportamiento no determinista de los tiempos aleatorios.

El recuento final de vocales fue idéntico en productor y consumidor:

Vocal	Producidas	Consumidas
a / A	8	8
e / E	6	6
i / I	8	8
o / O	5	5
u / U	3	3

2.6. Gestión de semáforos en el kernel

Los semáforos POSIX nombrados persisten en el kernel hasta que se eliminan explícitamente. Para evitar conflictos con ejecuciones anteriores, el productor llama a `sem_unlink()` *antes* de crear los semáforos y de nuevo *al terminar*:

```
/* Al inicio: eliminar posibles residuos de ejecuciones
   previas */
sem_unlink("VACIAS"); sem_unlink("LLENAS"); sem_unlink("
   MUTEX");

/* Crear e inicializar */
vacias = sem_open("VACIAS", O_CREAT, 0700, N);
llenas = sem_open("LLENAS", O_CREAT, 0700, 0);
mutex  = sem_open("MUTEX",  O_CREAT, 0700, 1);
```

```

/* ... bucle productor ... */

/* Al terminar: cerrar y eliminar del kernel */
sem_close(vacias); sem_close(llenas); sem_close(mutex);
sem_unlink("VACIAS"); sem_unlink("LLENAS"); sem_unlink("
    MUTEX");
```

El consumidor únicamente cierra sus descriptores con `sem_close()`, sin eliminar los semáforos del kernel.

2.7. Conclusiones

- Los semáforos POSIX resuelven de forma correcta y eficiente las tres causas de error del apartado anterior: `MUTEX` garantiza exclusión mutua; `VACIAS` bloquea al productor sin *busy-waiting*; `LLENAS` bloquea al consumidor sin *busy-waiting*.
- El orden de llamadas `sem_wait(condición)` → `sem_wait(mutex)` es fundamental para evitar interbloqueos. Invertirlo provocaría que un proceso retuviese el mutex mientras espera en el semáforo de condición, bloqueando al otro proceso indefinidamente.
- Las fases de velocidad demuestran visualmente que los semáforos de condición actúan como señales *sleep*/*wakeup*: el proceso lento queda suspendido sin consumir CPU hasta que el otro proceso libera un hueco o deposita un elemento.
- La coincidencia exacta de los conteos de vocales en todas las ejecuciones es la prueba objetiva de que no se ha perdido ni duplicado ningún carácter, independientemente de los *timings* aleatorios de la fase 3.
- La función `sem_unlink()` es imprescindible para una correcta limpieza de recursos del kernel; su omisión deja semáforos residuales que interfieren con ejecuciones posteriores.